키 관리 (Key Management)

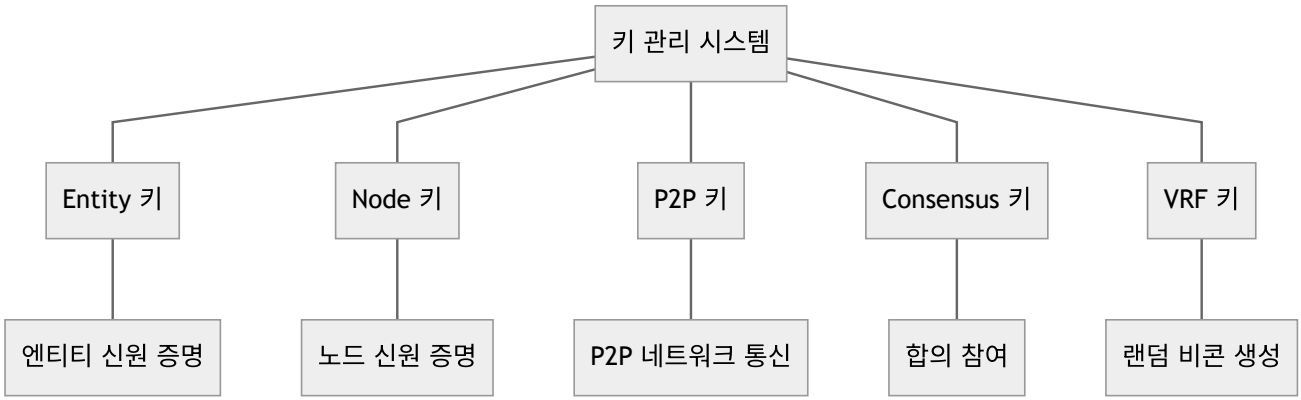
소개

Oasis Core의 키 관리 시스템은 네트워크의 보안을 담당하는 핵심 구성 요소입니다. 이 시스템은 노드 신원 확인, 암호화 통신, 디지털 서명 등 다양한 보안 기능을 위한 암호화 키의 생성, 저장, 배포, 회전을 관리합니다.

아키텍처 개요

키 유형별 분류

Oasis Core는 다음과 같은 역할별 키를 관리합니다:



키 역할 정의

```
// SignerRole은 서명자의 역할을 정의합니다

type SignerRole int
```

```go
const (
SignerEntity SignerRole = 1  // 엔티티 키
SignerNode SignerRole = 2  // 노드 키
SignerP2P SignerRole = 3  // P2P 통신 키
SignerConsensus SignerRole = 4  // 합의 키
SignerVRF SignerRole = 5  // VRF 키
)
```

노드 신원과 키 생성 프로세스

1. 키 생성 메커니즘

결정적 키 생성

```go
// 결정적 노드 키 생성 함수
func  GenerateDeterministicNodeKeys(dir *env.Dir, rawSeed  string, roles []signa
h := crypto.SHA512.New()
_, _ = h.Write([]byte(rawSeed))
var  publicKeys []signature.PublicKey
for  _, role := range  roles {
// 각 역할별로 고유한 키 생성
seed := h.Sum(nil)
privateKey := ed25519.NewKeyFromSeed(seed[:32])
publicKey := privateKey.Public().(ed25519.PublicKey)
```

```
publicKeys = append(publicKeys, signature.PublicKey(publicKey))
}
return  publicKeys, nil
}
```

엔트로피 기반 키 생성

```
// SignerFactory 인터페이스를 통한 키 생성
type SignerFactory interface {
// Generate는 새로운 개인키를 생성하고 저장합니다
Generate(role SignerRole, rng io.Reader) (Signer, error)
// Load는 저장된 개인키를 로드합니다
Load(role SignerRole) (Signer, error)
}
```

2. 키 생성 과정

1. **엔트로피 수집**: 안전한 난수 생성기를 통한 엔트로피 수집
2. **시드 생성**: SHA-512 해시를 통한 키 시드 생성
3. **키 쌍 생성**: Ed25519 알고리즘을 사용한 공개키/개인키 쌍 생성
4. **역할별 키 할당**: 각 SignerRole에 따른 키 할당
5. **키 검증**: 생성된 키의 유효성 검증

3. 노드 신원 확인

```go
// 노드 신원 확인을 위한 공개키 구조
type PublicKey struct {
ed25519.PublicKey
}


// 노드 등록 시 신원 증명
func (n *Node) RegisterIdentity(entityKey signature.PublicKey, nodeKey signature
// 엔티티 키로 노드 키에 서명
signature, err := entitySigner.ContextSign(RegistrationContext, nodeKey.MarshalB
if err != nil {
return err
}
// 레지스트리에 노드 등록
return registry.RegisterNode(nodeKey, signature)
}
```

키 저장과 보호 메커니즘

1. 파일 시스템 기반 저장

키 파일 구조

```
node_data/
```

```
├── identity/
| ├── entity.pem # 엔티티 개인키
| ├── entity_genesis.json # 엔티티 공개키 (제네시스용)
| ├── node.pem # 노드 개인키
| ├── node_genesis.json # 노드 공개키 (제네시스용)
| ├── p2p.pem # P2P 개인키
| ├── consensus.pem # 합의 개인키
| └── vrf.pem # VRF 개인키
└── tls/
├── cert.pem # TLS 인증서
└── key.pem # TLS 개인키
```

키 파일 보호

```go
// 키 파일 저장 시 권한 설정
func SavePrivateKey(keyPath string, privateKey []byte) error {
// 파일 권한을 소유자만 읽기/쓰기로 제한 (0600)
return os.WriteFile(keyPath, privateKey, 0600)
}

// 키 파일 로드 시 권한 확인
func LoadPrivateKey(keyPath string) ([]byte, error) {
info, err := os.Stat(keyPath)
if err != nil {
```

```go
return nil, err
}
// 파일 권한이 너무 개방적인지 확인
if info.Mode().Perm() > 0600 {
return nil, errors.New("private key file permissions too open")
}
return os.ReadFile(keyPath)
}
```

2. 메모리 보호

안전한 메모리 관리

```go
// Signer 인터페이스의 Reset 메서드로 민감한 데이터 삭제
type Signer interface {
Public() PublicKey
ContextSign(context Context, message []byte) ([]byte, error)
String() string
// Reset은 민감한 상태를 완전히 삭제합니다
Reset()
}

// 메모리 제로화 함수
func secureZero(data []byte) {
```

```go
for i := range data {
data[i] = 0
}
}
```

3. HSM (Hardware Security Module) 지원

```go
// HSM 기반 서명자 구현 예시
type HSMSigner struct {
keyID string
hsmConn HSMConnection
publicKey PublicKey
}

func (h *HSMSigner) ContextSign(context Context, message []byte) ([]byte, error)
// HSM 내부에서 서명 수행 (개인키가 HSM을 벗어나지 않음)
return h.hsmConn.Sign(h.keyID, context, message)
}
```

키 회전과 업데이트 절차

1. 키 회전 정책

자동 키 회전

```go
type KeyRotationPolicy struct {
RotationInterval time.Duration // 회전 주기
MaxKeyAge time.Duration // 최대 키 사용 기간
OverlapPeriod time.Duration // 새 키와 구 키 동시 사용 기간
}

// 키 회전 스케줄러
type KeyRotationScheduler struct {
policy KeyRotationPolicy
keyStore KeyStore
scheduler *time.Ticker
}

func (krs *KeyRotationScheduler) Start() {
krs.scheduler = time.NewTicker(krs.policy.RotationInterval)
go func() {
for range krs.scheduler.C {
if err := krs.rotateKeys(); err != nil {
log.Error("키 회전 실패", "error", err)
}
}
```

```
}()

}
```

2. 키 업데이트 프로세스

단계별 키 업데이트

```
func (km *KeyManager) UpdateKeys(newKeys  map[SignerRole]Signer) error {

// 1. 새 키 검증

for  role, signer := range  newKeys {

if  err := km.validateKey(role, signer);  err != nil {

return  fmt.Errorf("키 검증 실패 (role: %s): %w", role, err)

}

}

// 2. 기존 키 백업

if  err := km.backupCurrentKeys();  err != nil {

return  fmt.Errorf("키 백업 실패: %w", err)

}

// 3. 새 키 저장

for  role, signer := range  newKeys {

if  err := km.storeKey(role, signer);  err != nil {

// 실패 시 백업에서 복구

km.restoreFromBackup()

return  fmt.Errorf("키 저장 실패 (role: %s): %w", role, err)
```

```
}
}
// 4. 키 활성화
return km.activateNewKeys(newKeys)
}
```

3. 키 마이그레이션

네트워크 전체 키 업데이트

```
type KeyMigrationPlan struct {
MigrationEpoch uint64 // 마이그레이션 시작 에포크
NewKeys map[signature.PublicKey]signature.PublicKey // 구 키 -> 새 키 매핑
GracePeriod uint64 // 구 키 허용 기간
}

func (km *KeyManager) ExecuteMigration(plan KeyMigrationPlan) error {
currentEpoch := km.getCurrentEpoch()
switch {
case currentEpoch < plan.MigrationEpoch:
// 마이그레이션 준비 단계
return km.prepareMigration(plan)
case currentEpoch == plan.MigrationEpoch:
// 마이그레이션 실행
```

```
return km.performMigration(plan)
case currentEpoch <= plan.MigrationEpoch + plan.GracePeriod:
// 그레이스 피리어드 (구 키와 새 키 모두 허용)
return km.maintainGracePeriod(plan)
default:
// 구 키 완전 비활성화
return km.finalizeMigration(plan)
}
}
```

키 관리 API와 보안 설정

1. 키 관리 API

기본 키 관리 인터페이스

```
// KeyManager는 키 관리의 핵심 인터페이스입니다
type KeyManager interface {
// 키 생성
GenerateKey(role SignerRole) (Signer, error)
// 키 로드
LoadKey(role SignerRole) (Signer, error)
// 키 저장
StoreKey(role SignerRole, signer Signer) error
```

```go
// 키 삭제
DeleteKey(role SignerRole) error
// 키 목록 조회
ListKeys() ([]SignerRole, error)
// 키 상태 확인
GetKeyStatus(role SignerRole) (*KeyStatus, error)
}

type KeyStatus struct {
Role SignerRole `json:"role"`
PublicKey PublicKey `json:"public_key"`
CreatedAt time.Time `json:"created_at"`
LastUsed time.Time `json:"last_used"`
IsActive  bool  `json:"is_active"`
ExpiresAt *time.Time `json:"expires_at,omitempty"`
}
```

서명 컨텍스트 관리

```go
// 도메인 분리를 위한 컨텍스트 시스템
type Context string

// 새로운 서명 컨텍스트 생성
func  NewContext(rawContext  string, opts ...ContextOption) Context {
```

```go
// 컨텍스트 길이 검증 (RFC 8032 호환)
if len(rawContext) == 0 || len(rawContext) > ed25519.ContextMaxSize {
panic("잘못된 컨텍스트 길이")
}
return Context(rawContext)
}

// 체인별 도메인 분리
func WithChainSeparation() ContextOption {
return func(o *contextOptions) {
o.chainSeparation = true
}
}

// 컨텍스트 기반 서명
func (s *Signer) ContextSign(context Context, message []byte) ([]byte, error) {
// 컨텍스트와 메시지를 결합하여 서명
preparedMessage, err := PrepareSignerMessage(context, message)
if err != nil {
return nil, err
}
return s.sign(preparedMessage)
```

```
}
```

2. 보안 설정 예제

노드 설정 파일 예시

```yaml
# oasis-node 설정 파일
datadir: /opt/oasis/node


# 키 관리 설정
identity:
# 키 파일 경로
entity_key: /opt/oasis/node/identity/entity.pem
node_key: /opt/oasis/node/identity/node.pem
p2p_key: /opt/oasis/node/identity/p2p.pem
consensus_key: /opt/oasis/node/identity/consensus.pem
vrf_key: /opt/oasis/node/identity/vrf.pem
# 키 보안 설정
key_rotation:
enabled: true
interval: "720h"  # 30일마다 회전
backup_count: 5  # 백업 키 보관 개수
# HSM 설정 (선택사항)
hsm:
```

```yaml
enabled: false
provider: "pkcs11"
library_path: "/usr/lib/libpkcs11.so"
slot_id: 0
pin: "${HSM_PIN}"

# TLS 설정
tls:
cert_file: /opt/oasis/node/tls/cert.pem
key_file: /opt/oasis/node/tls/key.pem
client_auth: require_and_verify_client_cert
```

키 생성 스크립트 예시

```bash
#!/bin/bash
# 노드 키 생성 스크립트

set -euo pipefail

NODE_DIR="/opt/oasis/node"
IDENTITY_DIR="${NODE_DIR}/identity"

# 디렉토리 생성
mkdir -p "${IDENTITY_DIR}"
```

```
chmod 700 "${IDENTITY_DIR}"

# 각 역할별 키 생성
echo "엔티티 키 생성 중..."
oasis-node identity init-entity \
--entity.dir "${IDENTITY_DIR}" \
--entity.allow_entity_signed_nodes

echo "노드 키 생성 중..."
oasis-node identity init-node \
--node.dir "${IDENTITY_DIR}" \
--node.role validator

echo "P2P 키 생성 중..."
oasis-node identity init-p2p \
--p2p.dir "${IDENTITY_DIR}"

echo "합의 키 생성 중..."
oasis-node identity init-consensus \
--consensus.dir "${IDENTITY_DIR}"

echo "VRF 키 생성 중..."
oasis-node identity init-vrf \
```

```
--vrf.dir "${IDENTITY_DIR}"


# 파일 권한 설정
find "${IDENTITY_DIR}" -name "*.pem" -exec chmod 600 {} \;
find "${IDENTITY_DIR}" -name "*.json" -exec chmod 644 {} \;


echo "키 생성 완료!"
```

3. 키 검증 및 모니터링

키 상태 모니터링

```
type KeyMonitor struct {
keyManager KeyManager
metrics *KeyMetrics
logger *logging.Logger
}


func (km *KeyMonitor) MonitorKeys() {
ticker := time.NewTicker(1 * time.Minute)
defer ticker.Stop()
for range ticker.C {
roles, err := km.keyManager.ListKeys()
if err != nil {
```

```
km.logger.Error("키 목록 조회 실패", "error", err)
continue
}
for _, role := range roles {
status, err := km.keyManager.GetKeyStatus(role)
if err != nil {
km.logger.Error("키 상태 조회 실패", "role", role, "error", err)
continue
}
// 메트릭 업데이트
km.metrics.KeyAge.WithLabelValues(role.String()).Set(
time.Since(status.CreatedAt).Seconds(),
)
// 만료 임박 경고
if status.ExpiresAt != nil && time.Until(*status.ExpiresAt) < 24*time.Hour {
km.logger.Warn("키 만료 임박", "role", role, "expires_at", status.ExpiresAt)
}
}
}
}
```

보안 고려사항

1. 키 생성 보안

- **충분한 엔트로피**: 안전한 난수 생성기 사용 필수
- **시드 보호**: 키 생성 시드의 안전한 관리
- **타이밍 공격 방지**: 키 생성 시간의 일정성 유지

2. 키 저장 보안

- **파일 시스템 권한**: 개인키 파일의 엄격한 권한 관리 (0600)
- **메모리 보호**: 사용 후 민감한 데이터의 안전한 삭제
- **백업 암호화**: 키 백업 시 추가 암호화 적용

3. 키 사용 보안

- **컨텍스트 분리**: 서명 시 도메인별 컨텍스트 사용
- **재사용 방지**: 논스(nonce) 사용으로 재생 공격 방지
- **접근 제어**: 키 사용 권한의 세밀한 제어

4. 키 회전 보안

- **점진적 전환**: 새 키와 구 키의 점진적 전환
- **검증 절차**: 새 키의 철저한 검증
- **롤백 계획**: 문제 발생 시 안전한 롤백 절차

이러한 키 관리 시스템을 통해 Oasis Core는 네트워크 전체의 보안을 보장하며, 각 노드의 신원을 안전하게 관리할 수 있습니다.